An Internship Report

Of

THIRANEX IT SOLUTIONS – C++ PROGRAMMING

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech (Computer Science and Engineering)

Internship Role

Intern – C++ Programming

Internship Duration

17 August 2026 – 16 September 2026

By

Roll Number of Student: 25scs1003002884

Name of Student: Shaurya Shivhare

Batch: 2025-2029

CANDIDATE'S DECLARATION

I, Shaurya Shivhare, hereby declare that the internship report entitled "C++ Programming" is a record of the work carried out by me during my internship at Thiranex from 17 August 2026 to 16 September 2026.

During this internship, I worked in the role of Intern – C++ Programming and completed a series of practical software engineering projects involving Object-Oriented Programming (OOP), Data Structures & Algorithms (DSA), file stream persistence (I/O), memory management, Standard Template Library (STL), modular design, testing, and debugging.

The major projects completed during the internship were:

1. Student Record Management System — StudentVault
2. Bank Management System
3. Library Management System
4. Tic-Tac-Toe Game Engine

I declare that the work presented in this report is based on my internship activities and project implementations. The technical information included in the report has been prepared using the project source code, documentation, and development work completed during the internship.

I further declare that this report has been prepared for academic submission and has not been submitted elsewhere for the award of any other degree or qualification.

Name: Shaurya Shivhare

Roll No.: 25SCS1003002884

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Thiranex for providing me with the opportunity to undertake an internship in C++ Programming. This internship provided me with valuable practical exposure to the development of modular software systems and helped me understand how core computer science concepts apply to real-world applications.

I am thankful to the mentors and team members associated with the internship program for providing guidance and project requirements throughout the internship. The project-based nature of the internship allowed me to apply theoretical concepts in practical development scenarios.

I would also like to express my sincere gratitude to the faculty members and department of IILM University, Greater Noida for their guidance and support during the internship period and in the preparation of this report.

I am grateful to everyone who directly or indirectly supported me during this learning experience. The internship helped me strengthen my technical understanding, improve my problem-solving abilities, and gain practical experience in C++ software development.

Student Name: SHAURYA SHIVHARE

Roll No.: 25scs1003002884

TABLE OF CONTENTS

Chapter Title Page

1. INTRODUCTION

1.1 Overview of the Internship

The internship in C++ Programming at Thiranex was undertaken from 17 August 2026 to 16 September 2026 in a remote, project-based format. The internship provided practical exposure to software development by involving multiple projects with progressively different functional and technical requirements.

The primary focus of the internship was to understand and apply core concepts involved in C++ software engineering, including Object-Oriented Programming (OOP), Data Structures & Algorithms (DSA), file stream persistence (I/O), memory management, Standard Template Library (STL), modular compilation, terminal user interfaces, application testing, and debugging.

The internship consisted of four major development tasks:

1. Student Record Management System — StudentVault
2. Bank Management System
3. Library Management System
4. Tic-Tac-Toe Game Engine

The official internship documentation identifies the role as Intern – C++ Programming and the internship mode as Remote / Project-Based.

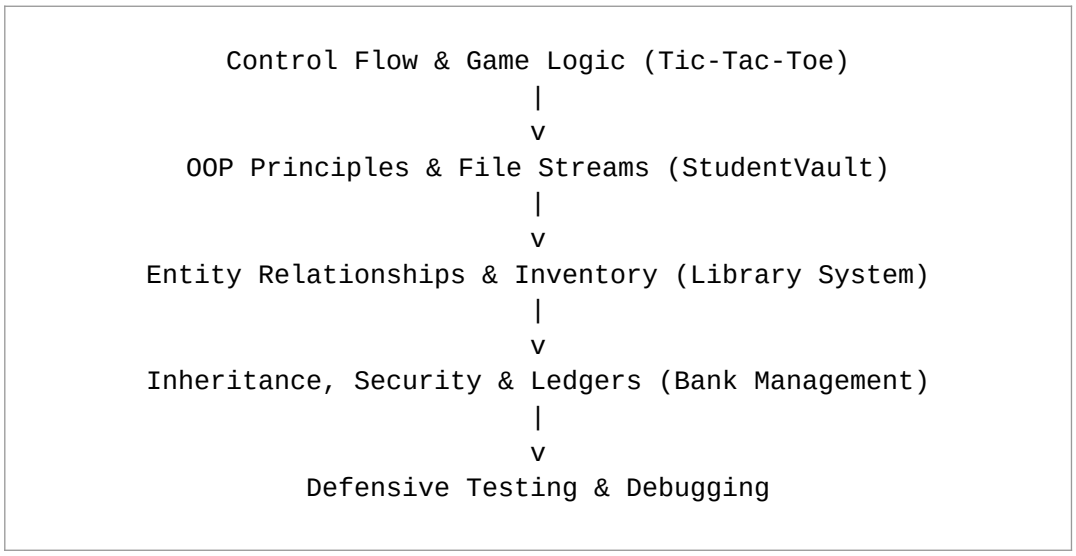
The internship was carried out according to the assigned timeline and deliverable specifications set out in the offer of internship.

1.2 Nature of the Internship

The internship followed a practical and project-oriented development approach. Each assigned project focused on a different application domain and introduced specific technical requirements.

The first project focused primarily on fundamental control flow, array manipulation, and game state logic. The subsequent projects introduced Object-Oriented class hierarchies, relational entity models, file stream serialization, input validation, and defensive exception handling.

The projects therefore provided progressive exposure to modern C++ software engineering:



This progression helped develop a broader understanding of how complete console and system software applications are designed and implemented.

2. ORGANIZATION PROFILE

2.1 Thiranex

Thiranex was the host organization through which the internship in C++ Programming was undertaken.

The internship was conducted in a remote/project-based format, with practical software development tasks assigned to the intern. The official offer documentation identifies the position as Intern – C++ Programming under candidate ID THX-AUG1726-265.

The internship program provided practical exposure to core software engineering through multiple structured milestones. These projects covered algorithm design, Object-Oriented architecture, data structure optimization, file handling, and version control workflows.

The project-based structure allowed the intern to work on applications of different types, beginning with an interactive algorithmic game and progressing toward data-intensive record systems, automated library circulation, and financial account ledgers.

Note: The report intentionally does not include unsupported claims about Thiranex's founding year, employee count, revenue, clients, or organizational size because those details are not established by the internship documents available for this report.

3. INTERNSHIP OBJECTIVES

The major objectives of the internship were:

3.1 Understanding Core C++ Architecture

To understand how memory management, pointers, references, and the standard compiler toolchain interact to build high-performance software.

3.2 Developing Practical Systems

To gain practical experience by designing and developing functional console applications based on real-world workflow requirements.

3.3 Object-Oriented Analysis & Design (OOAD)

To develop modular, scalable class hierarchies using Encapsulation, Inheritance, Polymorphism, and Abstraction.

3.4 Offline Data Persistence

To master persistent storage using C++ file streams (`ifstream`, `ofstream`, `fstream`) to serialize and deserialize data models.

3.5 Standard Template Library (STL)

To utilize industry-standard data structures such as `std::vector`, `std::map`, and algorithms (`std::sort`, `std::find`) for optimal execution time.

3.6 Defensive Programming & Validation

To implement input sanitization, handle non-numeric inputs in numeric fields, and prevent runtime crashes or infinite stream loops.

3.7 Software Testing and Debugging

To gain hands-on experience in isolating edge cases, testing boundary values, identifying logic flaws, and debugging memory lifetimes.

3.8 Version Control & Git Workflows

To manage source code repositories professionally using Git branching, structured commits, and GitHub project management.

4. PROJECTS COMPLETED DURING INTERNSHIP

During the internship, four major development projects were completed. Each project addressed a distinct problem domain in C++ software engineering.

4.1 STUDENT RECORD MANAGEMENT — STUDENTVAULT

4.1.1 Project Overview

StudentVault is a C++ academic management application built to maintain, update, query, and compute academic statistics for student cohorts. The project replaces error-prone manual paper registers with an automated console interface backed by persistent disk storage.

4.1.2 Objectives

- To build a robust student database using Object-Oriented class models.
- To implement full CRUD operations (Create, Read, Update, Delete) for student records.
- To compute student GPA/CGPA automatically from subject marks and credit weights.
- To serialize student data into persistent disk files.
- To provide fast search by Roll Number and Branch filters.

4.1.3 Technologies & Tools Used

- **Language:** C++17
- **Core Paradigms:** Object-Oriented Programming (OOP)
- **Libraries:** <iostream>, <fstream>, <vector>, <string>, <iomanip>
- **Tools:** GCC / MinGW, VS Code, Git, GitHub

4.1.4 Key Features

- Student registration with roll number, name, department, and contact.
- Automated semester GPA computation and academic standings.
- Search engine querying records by unique roll number.
- Update existing student demographic and course marks.
- Persistent storage using formatted text/binary file streams.
- Safe delete operations with confirmation checks.

4.1.5 Repository Link

GitHub: https://github.com/CodeWizard-1234/studentvault

4.2 BANK MANAGEMENT SYSTEM

4.2.1 Project Overview

The Bank Management System is a secure, console-driven financial transaction application that models core retail banking workflows. The system allows users to open accounts, perform real-time deposits and withdrawals, transfer money between accounts, and inspect detailed statement ledgers.

4.2.2 Objectives

- To simulate realistic financial transactions with account security.
- To implement inheritance hierarchies between Savings and Current account models.
- To enforce defensive checks against overdrafts, negative amounts, and invalid PINs.
- To maintain an immutable transaction history ledger saved to disk.

4.2.3 Technologies Used

- **Language:** C++
- **OOP Architecture:** Abstract Base Classes, Virtual Functions, Inheritance
- **File Storage:** Stream serialization for accounts and ledgers
- **Data Structures:** std::map for constant-time account lookups

4.2.4 Key Features

- Account Creation: Generate unique account numbers with initial balances.
- Deposit & Withdrawal: Atomically update balances with validation rules.
- Inter-Account Transfers: Transfer funds safely between two distinct account IDs.
- PIN Verification: Protect debit actions behind PIN authentication.
- Account Statement: Display structured chronological transaction histories.

4.2.5 Repository Link

GitHub: https://github.com/CodeWizard-1234/bank-management-system

4.3 LIBRARY MANAGEMENT SYSTEM

4.3.1 Project Overview

The Library Management System is an automated circulation and cataloging application. It manages the lifecycle of library inventory, tracking available book copies, active student borrowing records, return deadlines, and calculating penalty fines for late returns.

4.3.2 Objectives

- To build an automated catalog for adding, searching, and managing book inventories.
- To handle book issuance and returns with dynamic copy availability updates.
- To compute overdue fines based on days elapsed past the due date.
- To maintain persistent logs of both active and historical borrowings.

4.3.3 Technologies Used

- **Language:** C++
- **Design:** Modular entity relationships (Book, Member, Transaction)
- **STL:** std::vector, std::string, std::algorithm
- **Persistence:** Flat-file storage with custom delimiter parsing

4.3.4 Key Features

- Inventory Management: Add books with ISBN, title, author, and quantity.
- Issue Book: Verify copy availability and register student borrowing.
- Return Book: Process book returns and calculate late fines dynamically.
- Search Catalog: Fast lookup by Book ID or title substring matching.

4.3.5 Repository Link

GitHub: https://github.com/CodeWizard-1234/library-management-system-

4.4 TIC-TAC-TOE GAME ENGINE

4.4.1 Project Overview

The Tic-Tac-Toe Game Engine is an interactive, turn-based terminal application supporting both 2-Player local matches and a Single-Player mode against an intelligent computer AI. It features clean matrix state validation, move sanitization, and win/draw condition detection.

4.4.2 Objectives

- To master 2D array representation and game-loop architectures in C++.
- To implement deterministic algorithms for checking 8 winning combinations.
- To build an AI bot opponent capable of detecting threats and blocking winning moves.
- To design a clean, responsive terminal user interface.

4.4.3 Technologies Used

- **Language:** C++
- **Concepts:** 2D Arrays, Game Loops, Control Logic, Randomization

4.4.4 Key Features

- Dynamic 3x3 board rendering with row/column indices.
- Player vs Player & Player vs Computer AI gameplay modes.
- Validation against cell overwrites and out-of-boundary inputs.
- Scoreboard tracking victories, defeats, and ties across consecutive rounds.

4.4.5 Repository Link

GitHub: https://github.com/CodeWizard-1234/tic-tac-toe

5. TECHNOLOGIES AND TOOLS USED

5.1 Core Programming Language

C++ (C++11 / C++14 / C++17) was used as the foundational language across all four projects due to its strong type safety, high performance, deterministic memory management, and rich Standard Template Library.

5.2 Object-Oriented Paradigms

Classes, constructors, destructors, access specifiers (`public`, `private`, `protected`), operator overloading, and inheritance hierarchies were implemented to enforce separation of concerns and maintainable software architecture.

5.3 Standard Template Library (STL)

Used `std::vector` for dynamic storage, `std::map` for keyed lookups, `std::string` for robust text operations, and `std::algorithm` for sorting and searching.

5.4 File Streams (I/O)

Employed `std::ifstream`, `std::ofstream`, and `std::stringstream` to provide offline data persistence without requiring external database engines.

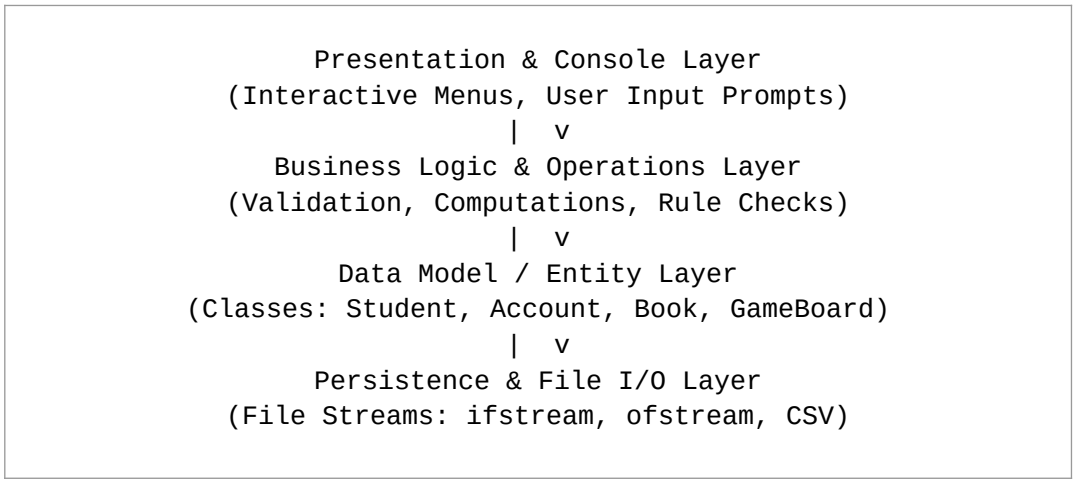
5.5 Development & Build Tools

- **Compiler:** GCC / G++ (MinGW on Windows)
- **IDE:** Visual Studio Code
- **Version Control:** Git & GitHub

6. SYSTEM ARCHITECTURE

6.1 Modular C++ Software Architecture

The C++ applications developed during the internship followed a structured, layered architectural model:



This separation ensures that user interface changes do not affect core business rules, and data structures can be serialized to disk independently of presentation.

7. DEVELOPMENT METHODOLOGY

An iterative engineering methodology was adopted throughout the 4-week internship:

7.1 Requirement Analysis

Each problem statement was analyzed to determine required entities, operations, constraints, and data flows.

7.2 Architecture & Class Design

Class diagrams, method signatures, and access boundaries were defined prior to implementation.

7.3 Implementation & Stream Persistence

Core algorithms and file serialization routines were written following modular C++ standards.

7.4 Verification & Testing

Applications were tested with nominal, boundary, and invalid inputs to guarantee stability.

8. TESTING, DEBUGGING AND CHALLENGES

8.1 Testing Coverage Areas

- **Boundary Values:** Minimum and maximum numeric balance transactions.
- **Empty Search Handling:** Querying nonexistent records without crashes.
- **Persistence Verification:** Verifying data consistency after immediate program restarts.
- **Concurrent File Handles:** Preventing file access locks during multiple operations.

8.2 Challenges and Solutions (Common Errors & Fixes)

Several recurring technical issues surfaced during development and were resolved as follows:

- **Stream Fail State:** Corrupted input streams after invalid entries were handled by invoking `cin.clear()` and `cin.ignore()` to flush corrupted buffers, preventing infinite input loops.
- **String Spacing:** Multi-word student names and book titles were being truncated at the first space; this was resolved by switching to `std::getline()` to correctly read multi-word input.

These fixes were consistent with the internship's broader objective of defensive programming and input validation (see Section 3.6).

9. LEARNING OUTCOMES

9.1 Technical Competencies Acquired

- Deepened proficiency in modern C++ features and idioms.
- Practical mastery of Object-Oriented Analysis & Design.
- Hands-on experience with STL containers and sorting/searching algorithms.
- Proficiency in low-level file serialization and persistent data models.

9.2 Academic Relevance to B.Tech Curriculum

The technical knowledge gained directly supports 3rd and 4th-semester engineering coursework, including Data Structures & Algorithms (DSA), Object-Oriented Programming, and Operating Systems.

10. OVERALL OUTCOME AND CONCLUSION

The C++ Programming Internship at Thiranex was a transformative learning experience. The hands-on development of four distinct applications provided comprehensive exposure to the software development lifecycle — from requirement specification to class design, algorithm implementation, file persistence, testing, and version control.

Through the projects — StudentVault, Bank Management System, Library Management System, and Tic-Tac-Toe — I gained practical confidence in writing clean, modular, and resilient C++ software that can serve as a strong technical foundation for future engineering coursework and industry endeavors.

11. SUPPORTING DOCUMENTS

11.1 Internship Offer Letter

Official selection and appointment document issued by Thiranex.

Thiranex Date: 17 August 2026

TO,

Shaurya Shivhare
Intern ID: THX-AUG1726-265

Subject: Offer Letter for Internship in C++ Programming

Dear Shaurya Shivhare,

Following our selection process, we are pleased to offer you an internship position at **Thiranex**. We believe your skills and enthusiasm will be a great addition to our team.

Internship Role	Intern – C++ Programming
Commencement Date	17 Aug 2026
Completion Date	16 Sept 2026
Work Mode	Remote / Project-Based

This is an electronically generated document for verification with Thiranex Verification Cell (www.thiranex.in).

12. REFERENCES & APPENDIX

12.1 Technical References

1. Bjarne Stroustrup, The C++ Programming Language (4th Edition), Addison-Wesley.
2. ISO/IEC C++ Standard Committee, C++17 Working Draft Standard.
3. CppReference Documentation — https://en.cppreference.com
4. Scott Meyers, Effective Modern C++, O'Reilly Media.
5. Git and GitHub Documentation — https://git-scm.com/doc

FINAL APPENDIX — PROJECT REPOSITORIES

Project Name	GitHub Repository Link
Student Record Management (StudentVault)	https://github.com/CodeWizard-1234/studentvault
Bank Management System	https://github.com/CodeWizard-1234/bank-management-system
Library Management System	https://github.com/CodeWizard-1234/library-management-system-
Tic-Tac-Toe Game Engine	https://github.com/CodeWizard-1234/tic-tac-toe

Shaurya Shivhare
B.Tech CSE • Roll No: 25scs1003002884 • IILM University, Greater Noida